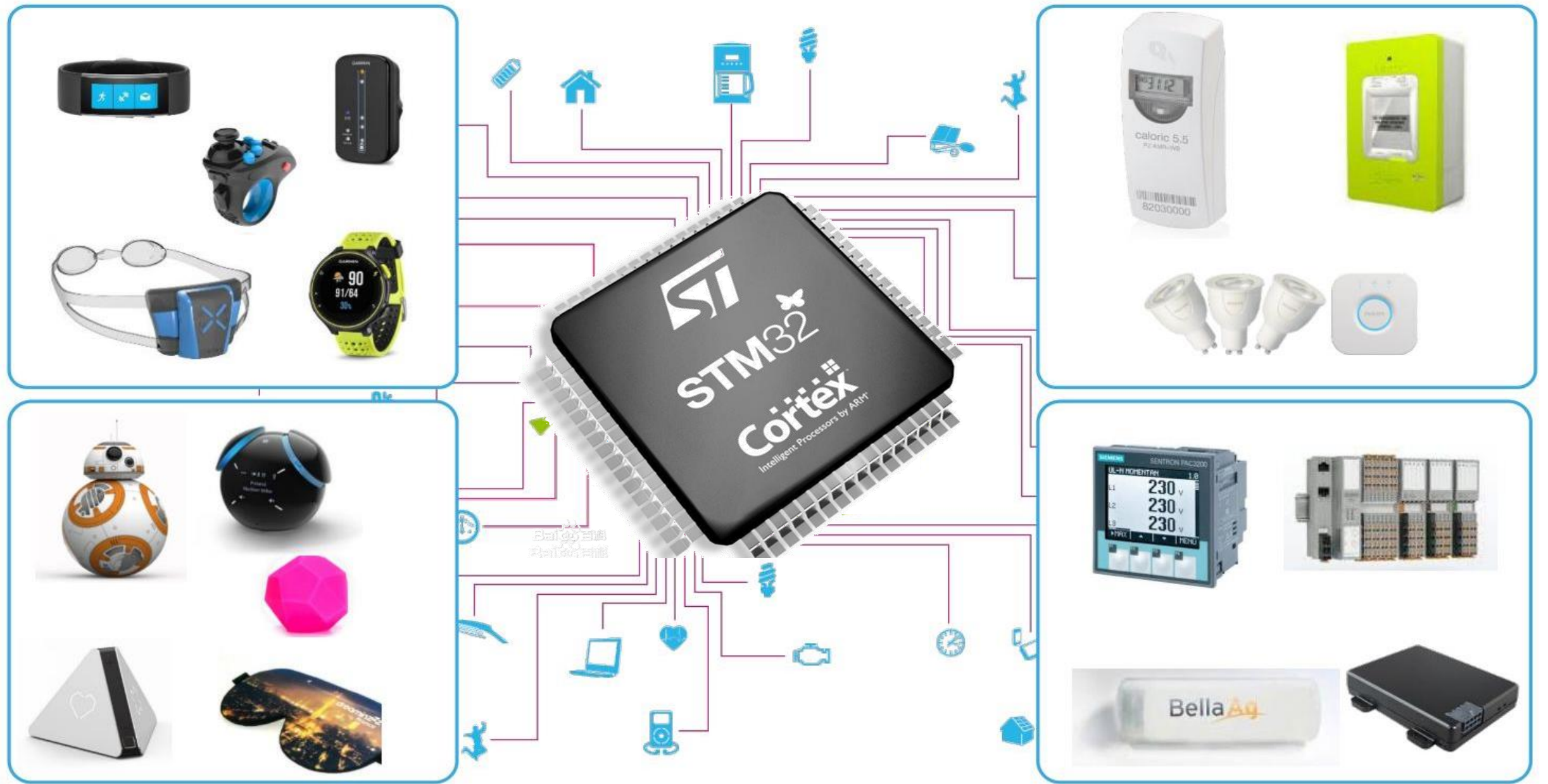




Instructor: WangXiaojing

Email: wangxj@sustech.edu.cn



Lecture2

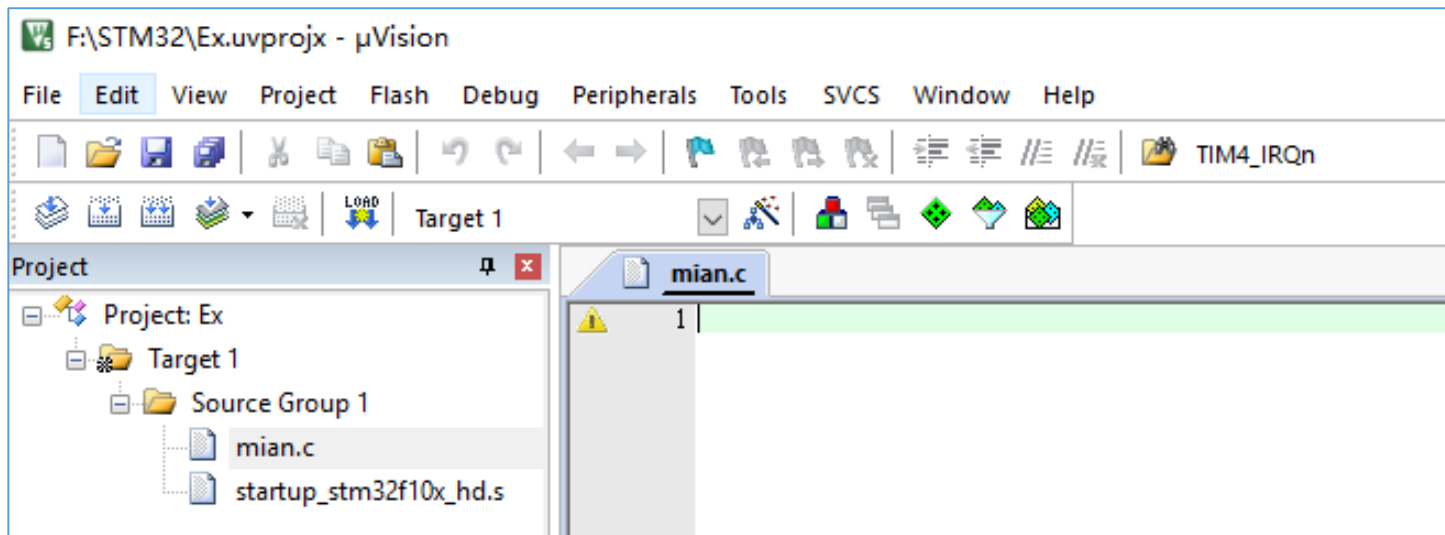
Register to Firmware Library

Topics

- **Create a new project**
- **Code Improvements**
 - from 0 to 1
 - from barbarism to civiliz
- **Firmware Library**

Part I: Create a new project

- Open the Keil5 software
- Project-> New μ Vision Project-> project name and Location(A new folder is better)
- Select Device->STM32F103VE-> Cancel
- File->New->Save(main.c)->Add Existing files to the Group....
- Copy the startup file to this folder,Add Existing files to the Group....



- Copy the following code to main.c-->Build(F7)
-  settings
- Build(F7)-> Download(F8)

```
int main (void)
```

```
{
```

```
// 打开 GPIOB 端口的时钟
```

```
*( unsigned int * )0x40021018 |= ( (1) << 3 );
```

```
// 配置IO口PB5为输出,推挽输出, 速率为10M
```

```
*( unsigned int * )0x40010C00 &= 0xFF0FFFFFF;
```

```
*( unsigned int * )0x40010C00 |= 0x00100000;
```

```
// 控制 ODR 寄存器
```

```
*( unsigned int * )0x40010C0C &= ~(1<<5); //LED on
```

```
/**( unsigned int * )0x40010C0C |= (1<<5); //LED off
```

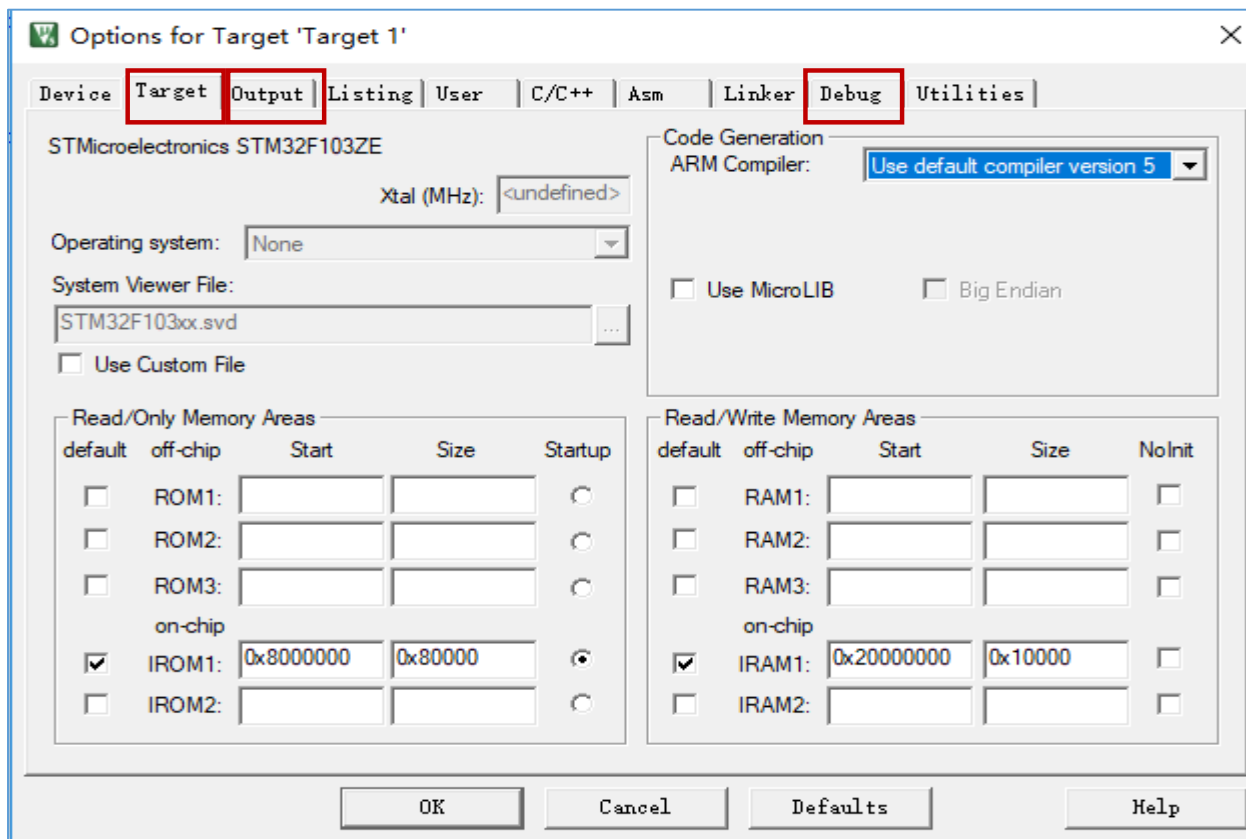
```
}
```

```
void SystemInit(void)
```

```
{
```

```
// 函数体为空, 目的是为了骗过编译器不报错
```

```
}
```



startup file

- The start-up file is necessary, and it is the entrance of the project;
- There is no need to modify, but to select the startup file matched with the chip used;
- Location: ..\官方固件库\STM32F10x_StdPeriph_Lib_V3.6.0 \Libraries \CMSIS\CM3 \DeviceSupport\ST\STM32F10x\startup\arm [Download URL](#)
- Which one: startup_stm32f10x_hd.s 

 startup_stm32f10x_cl.s

 startup_stm32f10x_hd.s

 startup_stm32f10x_hd_vl.s

 startup_stm32f10x_ld.s

 startup_stm32f10x_ld_vl.s

 startup_stm32f10x_md.s

 startup_stm32f10x_md_vl.s

 startup_stm32f10x_xl.s

启动文件	区别
startup_stm32f10x_ld.s	ld: low-density 小容量, FLASH 容量在 16-32K 之间
startup_stm32f10x_md.s	md: medium-density 中容量, FLASH 容量在 64-128K 之间
startup_stm32f10x_hd.s	hd: high-density 大容量, FLASH 容量在 256-512K 之间
startup_stm32f10x_xl.s	xl: 超大容量, FLASH 容量在 512-1024K 之间
以上四种都属于基本型, 包括 STM32F101xx、STM32F102xx、STM32F103xx 系列	
startup_stm32f10x_cl.s	cl:connectivity line devices 互联型, 特指 STM32F105xx 和 STM32F107xx 系列
startup_stm32f10x_ld_vl.s	vl:value line devices 超值型系列, 特指 STM32F100xx 系列
startup_stm32f10x_md_vl.s	
startup_stm32f10x_hd_vl.s	

Low-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 16 and 32 Kbytes.

Medium-density devices are STM32F101xx, STM32F102xx and STM32F103xx microcontrollers where the Flash memory density ranges between 64 and 128 Kbytes.

High-density devices are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 256 and 512 Kbytes.

XL-density devices are STM32F101xx and STM32F103xx microcontrollers where the Flash memory density ranges between 768 Kbytes and 1 Mbyte.

Connectivity line devices are STM32F105xx and STM32F107xx microcontrollers.

This section applies to low-, medium-, high- and XL-density STM32F10xxx devices. Connectivity line devices are dealt with in [Section 8: Connectivity line devices: reset and clock control \(RCC\)](#).



STM32 Standard Peripheral Libraries

STM32 Standard Peripheral Libraries Expansion

STM32 Standard Peripheral Drivers



■ STM32F37x/38x
■ STM32F301x/302x/303x/334x

<https://www.st.com/en/embedded-software/stm32-standard-peripheral-libraries.html>

<https://www.st.com/zh/embedded-software/stsw-stm32054.html>

startup_stm32f10x_hd.s

```
1  :***** (C) COPYRIGHT 2011 STMicroelectronics *****
2  : * File Name      : startup_stm32f10x_hd.s
3  : * Author         : MCD Application Team
4  : * Version        : V3.5.1
5  : * Date           : 08-September-2021
6  : * Description    : STM32F10x High Density Devices vector table for MDK-ARM
7  : *                : toolchain.
8  : *                : This module performs:
9  : *                : - Set the initial SP
10 : *                : - Set the initial PC == Reset_Handler
11 : *                : - Set the vector table entries with the exceptions ISR address
12 : *                : - Configure the clock system and also configure the external
13 : *                :   SRAM mounted on STM3210E-EVAL board to be used as data
14 : *                :   memory (optional, to be enabled by user)
15 : *                : - Branches to __main in the C library (which eventually
16 : *                :   calls main()).
17 : *                : After Reset the CortexM3 processor is in Thread mode,
18 : *                : priority is Privileged, and the Stack is set to Main.
19 : * <<< Use Configuration Wizard in Context Menu >>>
20 :*****
```

startup_stm32f10x_hd.s

```
147
148 ; Reset handler
149 Reset_Handler  PROC
150                 EXPORT Reset_Handler             [WEAK]
151                 IMPORT __main
152                 IMPORT SystemInit
153                 LDR     R0, =SystemInit
154                 BLX     R0
155                 LDR     R0, =__main
156                 BX      R0
157                 ENDP
158
```

Part II: Code Improvements---from 0 to 1 from barbarism to civiliz

Code Improvements

main.c

```
#define RCC_APB2ENR *(unsigned int*)(0x40021018)
#define GPIOB_CRL *(unsigned int*)(0x40010C00)
#define GPIOB_ODR *(unsigned int*)(0x40010C0C)
int main(void)
{
    //打开GPIOB端口的时钟
    RCC_APB2ENR|=((1)<<3);
    //配置IO口PB5为输出,推挽输出, 速率为10M
    GPIOB_CRL&=0xFF0FFFFFFF;
    GPIOB_CRL|=0X00100000;
    //控制ODR寄存器
    GPIOB_ODR&=~(1<<5);//LED on
    //GPIOB_ODR|=(1<<5);//LED off
}
```

stm32f10x.h

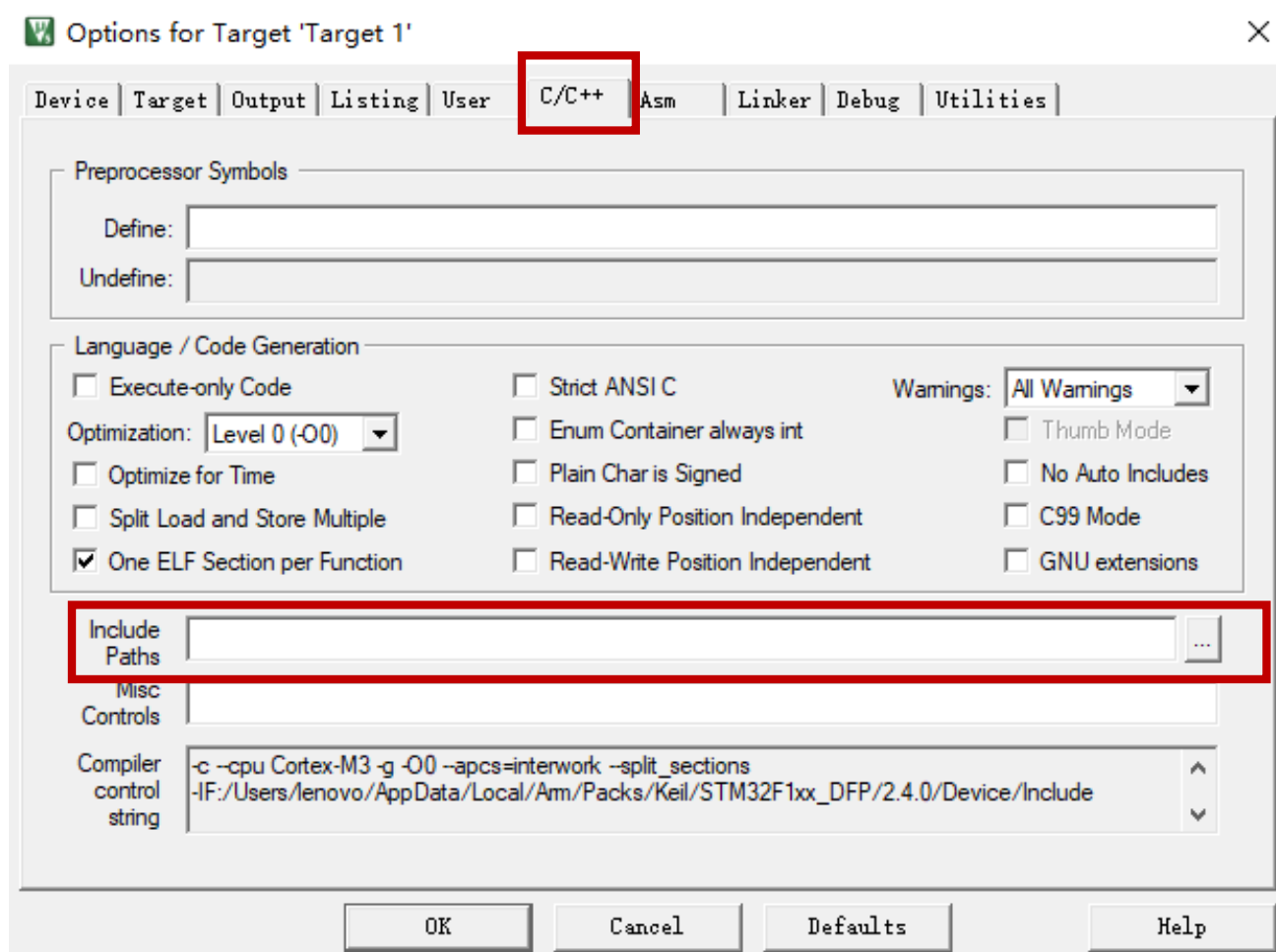
```
#define RCC_APB2ENR *(unsigned int*)(0x40021018)
#define GPIOB_CRL *(unsigned int*)(0x40010C00)
#define GPIOB_ODR *(unsigned int*)(0x40010C0C)
```

main.c

```
#include "stm32f10x.h"
int main(void)
{
    //打开GPIOB端口的时钟
    RCC_APB2ENR |= (1 << 3);
    //配置IO口PB5为输出,推挽输出, 速率为10M
    GPIOB_CRL &= 0xFF0FFFFFFF;
    GPIOB_CRL |= 0X00100000;
    //控制ODR寄存器
    GPIOB_ODR &= ~(1 << 5); //LED on
    //GPIOB_ODR |= (1 << 5); //LED off
}
```

#include<stm32f10x.h> vs #include"stm32f10x.h"

<xxx.h>,Keil自带的头文件，只要写上即可，无需指明路径，会去keil的安装目录里面找；
"xxx.h",自定义的头文件，需要指明路径；



stm32f10x.h

```
#define PERIPH_BASE((unsigned int)0x40000000)

#define APB1PERIPH_BASE PERIPH_BASE
#define APB2PERIPH_BASE (PERIPH_BASE+0x10000)
#define AHBPERIPH_BASE (PERIPH_BASE+0x20000)

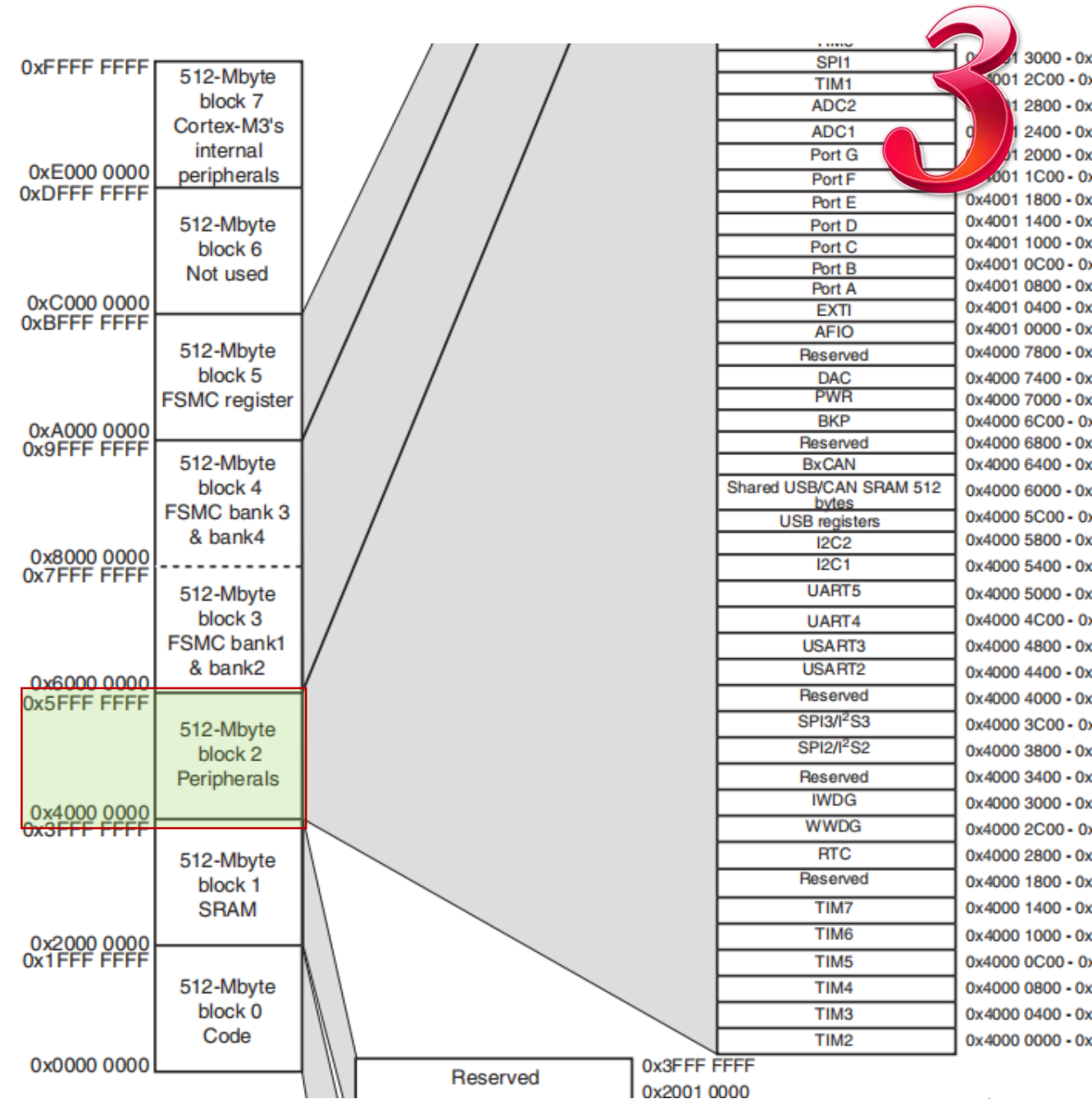
#define RCC_BASE (AHBPERIPH_BASE+0x1000)
#define GPIOA_BASE (APB2PERIPH_BASE+0x0800)
#define GPIOB_BASE (APB2PERIPH_BASE+0x0C00)

#define RCC_APB2ENR *((unsigned int*)(RCC_BASE+0x18)

#define GPIOA_CRL *((unsigned int*)(GPIOA_BASE+0x00)
#define GPIOA_CRH *((unsigned int*)(GPIOA_BASE+0x04)
#define GPIOA_ODR *((unsigned int*)(GPIOA_BASE+0x0C)
#define GPIOB_CRL *((unsigned int*)(GPIOB_BASE+0x00)
#define GPIOB_CRH *((unsigned int*)(GPIOB_BASE+0x04)
#define GPIOB_ODR *((unsigned int*)(GPIOB_BASE+0x0C)
```

main.c

write by yourself;



BUS Name	Bus Base Address
APB1	0x40000000
APB2	0x40010000
AHB	0x40018000

```
#define PERIPH_BASE ((unsignedint)0x40000000)
```

```
#define APB1PERIPH_BASE PERIPH_BASE
```

```
#define APB2PERIPH_BASE (PERIPH_BASE+0x10000)
```

```
#define AHBPERIPH_BASE (PERIPH_BASE+0x20000)
```

```
#define RCC_BASE (AHBPERIPH_BASE+0x1000)
```

```
#define GPIOA_BASE (APB2PERIPH_BASE+0x0800)
```

```
#define GPIOB_BASE (APB2PERIPH_BASE+0x0C00)
```

```
#define RCC_APB2ENR *(unsigned int*)(RCC_BASE+0x18)
```

```
#define GPIOA_CRL *(unsigned int*)(GPIOA_BASE+0x00)
```

```
#define GPIOA_CRH *(unsigned int*)(GPIOA_BASE+0x04)
```

```
#define GPIOA_ODR *(unsigned int*)(GPIOA_BASE+0x0C)
```

```
#define GPIOB_CRL *(unsigned int*)(GPIOB_BASE+0x00)
```

```
#define GPIOB_CRH *(unsigned int*)(GPIOB_BASE+0x04)
```

```
#define GPIOB_ODR *(unsigned int*)(GPIOB_BASE+0x0C)
```

Table 3. Register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 0000 - 0xA000 0FFF	FSMC	AHB	Section 21.6.9 on page 564
0x5000 0000 - 0x5003 FFFF	USB OTG FS		Section 28.16.6 on page 913
0x4003 0000 - 0x4FFF FFFF	Reserved		-
0x4002 8000 - 0x4002 9FFF	Ethernet		Section 29.8.5 on page 1071
0x4002 3400 - 0x4002 7FFF	Reserved		-
0x4002 3000 - 0x4002 33FF	CRC		Section 4.4.4 on page 65
0x4002 2000 - 0x4002 23FF	Flash memory interface		-
0x4002 1400 - 0x4002 1FFF	Reserved		-
0x4002 1000 - 0x4002 13FF	Reset and clock control RCC		Section 7.3.11 on page 121
0x4002 0800 - 0x4002 0FFF	Reserved		-
0x4002 0400 - 0x4002 07FF	DMA2		Section 13.4.7 on page 289
0x4002 0000 - 0x4002 03FF	DMA1		
0x4001 8400 - 0x4001 FFFF	Reserved		-
0x4001 8000 - 0x4001 83FF	SDIO		Section 22.9.16 on page 621
0x4001 2C00 - 0x4001 2FFF	TIM1 timer	APB2	Section 14.4.21 on page 363
0x4001 2800 - 0x4001 2BFF	ADC2		Section 11.12.15 on page 252
0x4001 2400 - 0x4001 27FF	ADC1		
0x4001 2000 - 0x4001 23FF	GPIO Port G		Section 9.5 on page 194
0x4001 1C00 - 0x4001 1FFF	GPIO Port F		
0x4001 1800 - 0x4001 1BFF	GPIO Port E		
0x4001 1400 - 0x4001 17FF	GPIO Port D		
0x4001 1000 - 0x4001 13FF	GPIO Port C		
0x4001 0C00 - 0x4001 0FFF	GPIO Port B		
0x4001 0800 - 0x4001 0BFF	GPIO Port A		Section 10.3.7 on page 214
0x4001 0400 - 0x4001 07FF	EXTI		
0x4001 0000 - 0x4001 03FF	AFIO		Section 9.5 on page 194

typedef unsigned int uint32_t;

```
typedef struct
{
    uint32_t CRL;
    uint32_t CRH;
    uint32_t IDR;
    uint32_t ODR;
    uint32_t BSRR;
    uint32_t BRR;
    uint32_t LCKR;
}GPIO_TypeDef;
```

- 9.2 GPIO registers
 - 9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)
 - 9.2.2 Port configuration register high (GPIOx_CRH) (x=A..G)
 - 9.2.3 Port input data register (GPIOx_IDR) (x=A..G)
 - 9.2.4 Port output data register (GPIOx_ODR) (x=A..G)
 - 9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)
 - 9.2.6 Port bit reset register (GPIOx_BRR) (x=A..G)
 - 9.2.7 Port configuration lock register (GPIOx_LCKR) (x=A..G)

Register	Start Address is Base
CRL	Base +0x00
CRH	Base +0x04
IDR	Base +0x08
ODR	Base +0x0C
BSRR	Base +0x10
BRR	Base +0x14
CLKR	Base +0x18

```
#define GPIOB ((GPIO_TypeDef*) GPIOB_BASE)
```



main.c

```
// 配置IO口为输出
GPIOB->CRL &= 0xFF0FFFFFF;
GPIOB->CRL |= 0x00100000;
// 控制 ODR 寄存器
GPIOB->ODR &= ~(1<<5);
//GPIOB->ODR |= (1<<5);
```



Modular Programming

Sub module

Source Files---xxx.c

Head Files --- xxx.h

main.c

// 打开 GPIOB 端口的时钟

RCC->APB2ENR |= (1) << 3 ;

// 配置IO口为输出

GPIOB->CRL &= 0xFF0FFFFFF;

GPIOB->CRL |= 0x00100000;

// 控制 ODR 寄存器

GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on

//GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off

stm32f10x_gpio.c

```
1 #include "stm32f10x_gpio.h"
2
3 void GPIO_SetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
4 {
5     GPIOx->BSRR |= GPIO_Pin;
6 }
7
8 void GPIO_ResetBits( GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin )
9 {
10     GPIOx->BRR |= GPIO_Pin;
11 }
```

stm32f10x_gpio.h

```
1 #ifndef __STM32F10X_GPIO_H
2 #define __STM32F10X_GPIO_H
3
4 #include "stm32f10x.h"
5
6 #define GPIO_Pin_0 ((uint16_t)0x0001) /*!< 选择Pin0 */ // (00000000 00000001)b
7 #define GPIO_Pin_1 ((uint16_t)0x0002) /*!< 选择Pin1 */ // (00000000 00000010)b
8 #define GPIO_Pin_2 ((uint16_t)0x0004) /*!< 选择Pin2 */ // (00000000 00000100)b
9 #define GPIO_Pin_3 ((uint16_t)0x0008) /*!< 选择Pin3 */ // (00000000 00001000)b
10 #define GPIO_Pin_4 ((uint16_t)0x0010) /*!< 选择Pin4 */ // (00000000 00010000)b
11 #define GPIO_Pin_5 ((uint16_t)0x0020) /*!< 选择Pin5 */ // (00000000 00100000)b
12 #define GPIO_Pin_6 ((uint16_t)0x0040) /*!< 选择Pin6 */ // (00000000 01000000)b
13 #define GPIO_Pin_7 ((uint16_t)0x0080) /*!< 选择Pin7 */ // (00000000 10000000)b
14
15 #define GPIO_Pin_8 ((uint16_t)0x0100) /*!< 选择Pin8 */ // (00000001 00000000)b
16 #define GPIO_Pin_9 ((uint16_t)0x0200) /*!< 选择Pin9 */ // (00000010 00000000)b
17 #define GPIO_Pin_10 ((uint16_t)0x0400) /*!< 选择Pin10 */ // (00000100 00000000)b
18 #define GPIO_Pin_11 ((uint16_t)0x0800) /*!< 选择Pin11 */ // (00001000 00000000)b
19 #define GPIO_Pin_12 ((uint16_t)0x1000) /*!< 选择Pin12 */ // (00010000 00000000)b
20 #define GPIO_Pin_13 ((uint16_t)0x2000) /*!< 选择Pin13 */ // (00100000 00000000)b
21 #define GPIO_Pin_14 ((uint16_t)0x4000) /*!< 选择Pin14 */ // (01000000 00000000)b
22 #define GPIO_Pin_15 ((uint16_t)0x8000) /*!< 选择Pin15 */ // (10000000 00000000)b
23 #define GPIO_Pin_All ((uint16_t)0xFFFF) /*!< 选择全部引脚 */ // (11111111 11111111)b
24
25 void GPIO_SetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
26 void GPIO_ResetBits( GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin );
27
28 #endif /* __STM32F10X_GPIO_H */
29
```

Conditional compilation: to prevent repeated inclusion of files

Imitating FirmWare-library Programming



main.c

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register
General purpose output	Push-pull	0	0	01 10 11 see Table 21		0 or 1
	Open-drain		1			0 or 1
Alternate Function output	Push-pull	1	0			Don't care
	Open-drain		1			Don't care
Input	Analog	0	0	00		Don't care
	Input floating		1			Don't care
	Input pull-down	1	0			0
	Input pull-up					1

Table 21. Output MODE bits

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

```
// 打开 GPIOB 端口的时钟
```

```
RCC->APB2ENR |= ( (1) << 3 );
```

```
// 配置IO口为输出
```

```
GPIO_InitTypeDef GPIO_InitStructure;
```

```
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;
```

```
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
```

```
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
```

```
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

```
// 控制 ODR 寄存器
```

```
GPIO_ResetBits(GPIOB,GPIO_Pin_5); //LED on
```

```
//GPIO_SetBits(GPIOB,GPIO_Pin_5); //LED off
```

```
typedef struct
{
    uint16_t GPIO_Pin;
    uint16_t GPIO_Speed;
    uint16_t GPIO_Mode;
}GPIO_InitTypeDef;
```

```
#define GPIO_Speed_10MHz ((uint16_t)0x0001)    // 10MHz    (01)b
#define GPIO_Speed_2MHz  ((uint16_t)0x0002)    // 2MHz     (10)b
#define GPIO_Speed_50MHz ((uint16_t)0x0003)    // 50MHz    (11)b

#define GPIO_Mode_AIN      ((uint16_t)0x0000) // 模拟输入  (0000 0000)b
#define GPIO_Mode_IN_FLOATING ((uint16_t)0x0004) // 浮空输入  (0000 0100)b
#define GPIO_Mode_IPD      ((uint16_t)0x0028) // 下拉输入  (0010 1000)b
#define GPIO_Mode_IPU      ((uint16_t)0x0048) // 上拉输入  (0100 1000)b

#define GPIO_Mode_Out_OD    ((uint16_t)0x0014) // 开漏输出  (0001 0100)b
#define GPIO_Mode_Out_PP    ((uint16_t)0x0010) // 推挽输出  (0001 0000)b
#define GPIO_Mode_AF_OD    ((uint16_t)0x001c) // 复用开漏输出 (0001 1100)b
#define GPIO_Mode_AF_PP    ((uint16_t)0x0018) // 复用推挽输出 (0001 1000)b
```

```
typedef struct
{
    uint16_t GPIO_Pin;
    GPIOSpeed_TypeDef GPIO_Speed;
    GPIO_Mode_TypeDef GPIO_Mode;
}GPIO_InitTypeDef;
```

```
typedef enum
{
    GPIO_Speed_10MHz = 1,    // 10MHz    (01)b
    GPIO_Speed_2MHz,        // 2MHz    (10)b
    GPIO_Speed_50MHz        // 50MHz    (11)b
}GPIOSpeed_TypeDef;

typedef enum
{
    GPIO_Mode_AIN = 0x0,    // 模拟输入    (0000 0000)b
    GPIO_Mode_IN_FLOATING = 0x04, // 浮空输入    (0000 0100)b
    GPIO_Mode_IPD = 0x28,    // 下拉输入    (0010 1000)b
    GPIO_Mode_IPU = 0x48,    // 上拉输入    (0100 1000)b

    GPIO_Mode_Out_OD = 0x14, // 开漏输出    (0001 0100)b
    GPIO_Mode_Out_PP = 0x10, // 推挽输出    (0001 0000)b
    GPIO_Mode_AF_OD = 0x1C, // 复用开漏输出 (0001 1100)b
    GPIO_Mode_AF_PP = 0x18 // 复用推挽输出 (0001 1000)b
}GPIO_Mode_TypeDef;
```



Homework of lecture2

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_InitStructure.GPIO_Pin= GPIO_Pin_5;  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;  
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

Firmware library programming



```
GPIOB->CRL &= 0xFF0FFFFFF;  
GPIOB->CRL |= 0x00100000;
```

Register Programming

Question

Configure PC13 as a pull-up input pin by 2 method refer to the example above,

- 1: Firmware library programming
- 2: Register Programming



Supplement 1—Typedef of C language

- Typedef is a key word of C language.
- It is used to define a new name for a data type.

- `typedef unsigned int uint32_t;`

- `typedef struct`

- `{`

- `uint32_t CRL;`

- `uint32_t CRH;`

- `uint32_t IDR;`

- `uint32_t ODR;`

- `....`

- `}GPIO_TypeDef;`

Supplement 2—Struct of C language

- A structure is a collection that contains multiple variables or arrays. Their types can be the same or different.

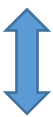
Each such variable or array is called a member of the structure.

- The members of the structure are arranged in sequence in memory;

- How to reference the structure variable---two methods

---' .' eg: GPIO_TypeDef AAA; AAA.CRL

---' -> ' eg: GPIO_TypeDef *AAA; AAA->CRL


(*AAA) .CRL

```
typedef struct
{
    uint32_t CRL;
    uint32_t CRH;
    uint32_t IDR;
    uint32_t ODR;
    uint32_t BSRR;
    uint32_t BRR;
    uint32_t LCKR;
}GPIO_TypeDef;
```

```
#define GPIOB ((GPIO_TypeDef*)GPIOB_BASE)
```



Supplement 3—enum of C language

- The enumeration members in { } are constants rather than variables. They cannot be assigned values, and their values can only be assigned to other variables.
- An enumeration type variable can only be assigned with its own enumeration member

```
typedef struct
{
    uint16_t GPIO_Pin;
    GPIOSpeed_TypeDef GPIO_Speed;
    GPIOMode_TypeDef GPIO_Mode;
}GPIO_InitTypeDef;
```

```
typedef enum
{
    GPIO_Speed_10MHz = 1,
    GPIO_Speed_2MHz,
    GPIO_Speed_50MHz
}GPIOSpeed_TypeDef;
```



```
typedef enum
{
    GPIO_Speed_10MHz = 1,
    GPIO_Speed_2MHz = 2,
    GPIO_Speed_50MHz = 3
}GPIOSpeed_TypeDef;
```



- 1、在没有显示说明的情况下，枚举常量（也就是花括号中的常量名）默认第一个枚举常量的值为0，往后每个枚举常量依次递增1.
- 2、在部分显示说明的情况下，未指定的枚举名的值将依着之前最有一个指定值向后依次递增